Point-in-Time Recovery (PITR)

Table of Contents

Learning Objectives

By the end of this module, you will be able to:

- Explain how PITR works conceptually and technically
- Configure WAL archiving as the foundation of PITR
- Perform a complete PITR recovery to a specific timestamp
- Use recovery target options (time, XID, LSN, name)
- Recover from accidental data deletion with minimal data loss
- Test PITR without affecting production

What is PITR and Why It Matters

Point-in-Time Recovery (PITR) allows you to restore a PostgreSQL database to any specific moment in the past, not just to the last backup.

```
WHY PITR IS ESSENTIAL:

Scenario: Developer runs "DELETE FROM orders WHERE status = 'old';"
at 14:32 on Tuesday, intending to delete 100 rows.
Bug: WHERE clause incorrect → DELETES ALL 5 MILLION rows.
Discovered: 14:45 on Tuesday.

Without PITR:
  Last backup: Monday 02:00
  Data loss: ~37 hours of orders!

With PITR:
  Restore to 14:31 on Tuesday
  Data loss: ~1 minute (since last WAL segment was archived)
```

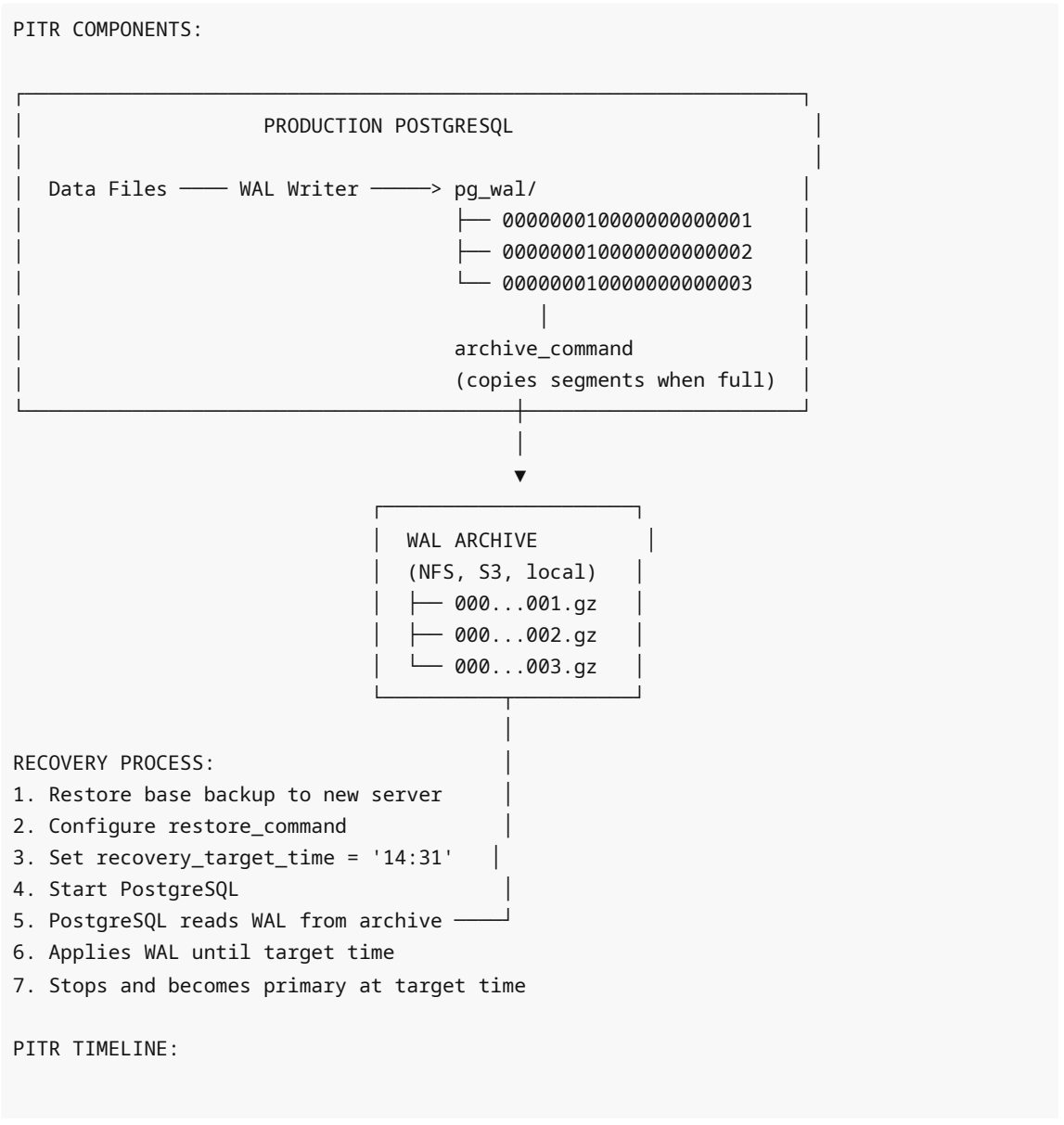
```
This is why PITR is non-negotiable for production PostgreSQL.
```

PITR vs. Standard Restore

Feature	Standard Restore	PITR
Recovery point	Backup time only	Any time after backup
Requires WAL	No	Yes (archived)
Setup complexity	Low	Medium
Protection against	Server failure	Server failure + logical errors

---

PITR Architecture



```
Backup (02:00) ──WAL──> 14:31 ──STOP──> recovery complete
                                          "data as of 14:31"
```

Prerequisites: WAL Archiving

PITR requires continuous WAL archiving. Every WAL segment must be archived.

```
# postgresql.conf — Required settings for PITR

# Enable WAL archiving
archive_mode = on
archive_command = 'test ! -f /archive/%f && cp %p /archive/%f'
# %p = full path of WAL file to be archived
# %f = filename only

# WAL level
wal_level = replica      # Minimum for PITR
# or: logical (if also doing logical replication)

# Checkpoint settings (affect how current archive is)
archive_timeout = 300    # Force WAL switch every 5 minutes
                         # Even for idle databases: archive is <= 5 min behind
                         # Default: 0 (no timeout, wait for segment to fill at 16MB)
```

Important: Changes to `archive_mode` require a PostgreSQL restart.

Configuration: archive_command

The `archive_command` is a shell command that PostgreSQL runs to archive each WAL segment. It MUST exit 0 on success.

```
# EXAMPLE 1: Simple local copy (development/test)
archive_command = 'test ! -f /archive/%f && cp %p /archive/%f'
# - test ! -f: Only copy if file doesn't already exist (idempotency)
# - cp %p %f: Copy from pg_wal to archive directory

# EXAMPLE 2: Copy to NFS
archive_command = 'test ! -f /mnt/nfs/pg_archive/%f && cp %p /mnt/nfs/pg_archive/%f'

# EXAMPLE 3: Copy to S3 (via aws cli)
archive_command = 'aws s3 cp %p s3://my-pg-archive/%f'
# Note: aws s3 cp is idempotent by default

# EXAMPLE 4: Copy to S3 with compression
archive_command = 'gzip -c %p | aws s3 cp - s3://my-pg-archive/%f.gz'
# restore_command must decompress: aws s3 cp s3://.../%f.gz - | gunzip > %p

# EXAMPLE 5: rsync to remote server
archive_command = 'rsync -a %p backup-server:/archive/wal/%f'
```

```
# EXAMPLE 6: pgBackRest (recommended for production)
archive_command = 'pgbackrest --stanza=mydb archive-push %p'

# EXAMPLE 7: WAL-G
archive_command = 'wal-g wal-push %p'

# Test archive_command manually
# Replace %p with an actual WAL file path, %f with its filename
cp /var/lib/postgresql/16/main/pg_wal/000000010000000000000001 /tmp/test_wal_file
test ! -f /archive/000000010000000000000001 && cp /tmp/test_wal_file
/archive/000000010000000000000001
echo $?  # Must be 0
```

Verify Archiving is Working

```
-- Check archive activity
SELECT archived_count, failed_count, last_archived_wal,
       last_archived_time, last_failed_wal, last_failed_time
FROM pg_stat_archiver;

-- Check if archiving is keeping up
SELECT pg_walfile_name(pg_current_wal_insert_lsn()) AS current_wal,
       last_archived_wal,
       pg_walfile_name(pg_current_wal_insert_lsn()) = last_archived_wal AS
is_current
FROM pg_stat_archiver;
```

Taking the Base Backup

```
# Before PITR recovery is possible, you need a base backup.
# The base backup + all WAL from backup time = PITR capability.

# Take base backup (record the backup start time!)
pg_basebackup \
    --host=localhost \
    --username=backupuser \
    --pgdata=/backup/base/$(date +%Y%m%d_%H%M%S) \
    --wal-method=stream \
    --checkpoint=fast \
    --label="pitr-base-$(date +%Y%m%d)" \
    --progress

# Record when backup completed (important for knowing recovery range)
echo "Base backup completed at: $(date)" > /backup/base/backup_info.txt

# The backup_label file inside the backup tells you the start LSN
cat /backup/base/20240115_020000/backup_label
# START WAL LOCATION: 0/3000000
# CHECKPOINT LOCATION: 0/3000028
```

```
# BACKUP METHOD: streamed
# BACKUP FROM: primary
# START TIME: 2024-01-15 02:00:00 UTC
# LABEL: pitr-base-20240115
```

Recovery Configuration

After restoring the base backup, configure recovery in `postgresql.conf` (PostgreSQL 12+).

```
# postgresql.conf (or postgresql.auto.conf) on the RECOVERY server

#------------------------------------------------------------
# PITR RECOVERY SETTINGS
#------------------------------------------------------------

# restore_command: How to get WAL files from archive
# This is called for each WAL file needed during recovery
restore_command = 'cp /archive/%f %p'
# or from S3:
# restore_command = 'aws s3 cp s3://my-pg-archive/%f %p'
# or from pgBackRest:
# restore_command = 'pgbackrest --stanza=mydb archive-get %f %p'

# Recovery target: WHEN to stop recovery
# (see next section for all options)
recovery_target_time = '2024-01-15 14:31:00 UTC'

# What to do after reaching the target
recovery_target_action = 'promote'
# Options:
# 'pause'   : Pause at target; inspect data; manually promote
# 'promote' : Automatically become primary (common for automated recovery)
# 'shutdown': Shutdown after reaching target (for verification)

# Which timeline to follow
# 'latest' = follow the most recent timeline (default for standbys)
# '1' = follow the original timeline
recovery_target_timeline = 'latest'

# Optional: Set to avoid replaying the target point transaction itself
# recovery_target_inclusive = true  # (default) replay up to and including target
# recovery_target_inclusive = false # stop just before target
```

Trigger Recovery

```
# In PostgreSQL 12+: no recovery.conf file
# Settings go in postgresql.conf or postgresql.auto.conf
# Start recovery with standby.signal file:
```

```
touch /var/lib/postgresql/16/main/standby.signal
# No, wait - for PITR recovery (not streaming replication standby):
# DO NOT create standby.signal for pure PITR recovery

# For PITR (restore to specific time, then become primary):
# Just set recovery settings in postgresql.conf
# Do NOT create standby.signal (that's for streaming standby)
# PostgreSQL detects pg_wal/RECOVERY file or backup_label and enters recovery

# Actually in PG12+:
# - standby.signal: enters standby/streaming mode (continuous recovery)
# - backup_label file in the backup: pg_basebackup creates this
#   PostgreSQL finds backup_label and enters archive recovery mode
```

---

recovery_target Options

```
# OPTION 1: Time-based (most common for "oops, I deleted the data")
recovery_target_time = '2024-01-15 14:31:00 UTC'
# Replay WAL until this timestamp

# OPTION 2: Transaction ID
recovery_target_xid = '1234567'
# Replay WAL through the transaction with this XID
# SELECT txid_current(); shows current XID
# Find the XID from audit logs or WAL analysis

# OPTION 3: LSN (Log Sequence Number)
recovery_target_lsn = '0/15D4B00'
# Precise replay control at the WAL position level

# OPTION 4: Named restore point
# On primary (before anticipated risky operation):
SELECT pg_create_restore_point('before_bulk_delete');
-- Returns: (0/15D4B00) ← the LSN of the restore point

# In recovery configuration:
recovery_target_name = 'before_bulk_delete'
# Replays to that named point

# OPTION 5: Immediate (stop as soon as database is consistent)
recovery_target = 'immediate'
# Used when you just want to get to a consistent state
# (e.g., recover from cluster crash, not specifically to a time)
```

Complete PITR Walkthrough

Scenario: Accidental `DELETE FROM orders` at 14:32 on 2024-01-15. Need to recover to 14:31.

Step 1: Assess the Situation

```
# On production (if still accessible):
psql -c "SELECT COUNT(*) FROM orders;"  # Should show 0 or abnormal count

# Find when the delete happened:
# Check audit logs
# Check PostgreSQL logs: grep "DELETE FROM orders" /var/log/postgresql/*.log

# Determine recovery target time (just before the delete)
# From logs: DELETE executed at 2024-01-15 14:32:15 UTC
# Recovery target: 2024-01-15 14:31:00 UTC (1 minute before)
```

Step 2: Prepare Recovery Server

```
# Option A: Recover on a SEPARATE server (safest — doesn't touch production)
# Option B: Recover in place (risky — overwrites production)
# ALWAYS prefer Option A for safety

# On recovery server:
# 1. Install same PostgreSQL version
sudo apt-get install -y postgresql-16

# 2. Stop PostgreSQL
sudo systemctl stop postgresql@16-main
sudo -u postgres rm -rf /var/lib/postgresql/16/main/*

# 3. Restore base backup
sudo -u postgres tar -xzf /backup/base/20240115_020000/base.tar.gz \
    -C /var/lib/postgresql/16/main/
sudo -u postgres tar -xzf /backup/base/20240115_020000/pg_wal.tar.gz \
    -C /var/lib/postgresql/16/main/pg_wal/

# Verify backup_label exists (triggers archive recovery)
cat /var/lib/postgresql/16/main/backup_label
```

Step 3: Configure Recovery

```
# Edit postgresql.conf on recovery server
sudo -u postgres cat >> /var/lib/postgresql/16/main/postgresql.conf << 'EOF'

# PITR Recovery Configuration
restore_command = 'cp /archive/%f %p'
recovery_target_time = '2024-01-15 14:31:00 UTC'
recovery_target_action = 'pause'    # Pause first to verify data
recovery_target_timeline = '1'      # Follow original timeline
EOF

# Ensure archive is accessible from recovery server
```

```
# (Mount NFS, configure S3 access, etc.)
ls /archive/  # Should see WAL files
```

Step 4: Start Recovery

```
# Start PostgreSQL in recovery mode
sudo systemctl start postgresql@16-main

# Monitor recovery progress (follow logs)
sudo tail -f /var/log/postgresql/postgresql-16-main.log

# Expected log output:
# LOG:  starting archive recovery
# LOG:  restored log file "000000010000000000000001" from archive
# LOG:  redo starts at 0/3000028
# LOG:  consistent recovery state reached at 0/3000100
# LOG:  database system is ready to accept read-only connections
# ...
# LOG:  recovery stopping before commit of transaction 123456, time 2024-01-15
14:32:15
# LOG:  pausing at the end of recovery
# HINT: Execute pg_wal_replay_resume() to promote.
```

Step 5: Verify Data

```
-- Connect to recovery server (read-only while paused)
psql -h recovery-server -U postgres -d mydb

-- Verify data is intact
SELECT COUNT(*) FROM orders;
-- Should show the full 5 million rows!

-- Check what you would lose (rows added between 14:31 and failure)
SELECT COUNT(*) FROM orders WHERE created_at > '2024-01-15 14:31:00';
-- These rows are from the "future" relative to recovery point
-- After promotion, they will not exist

-- Spot check critical data
SELECT id, user_id, total, created_at FROM orders ORDER BY created_at DESC LIMIT 10;
```

Step 6: Promote and Export

```
-- If data looks correct: promote to primary
SELECT pg_wal_replay_resume();
-- or: pg_ctl promote -D /var/lib/postgresql/16/main

-- Verify promotion
SELECT pg_is_in_recovery();  -- Returns 'f' (false = primary)
```

```
# Export recovered data back to production
pg_dump -h recovery-server -U postgres -d mydb -t orders -F c -f
orders_recovered.dump

# Restore to production
pg_restore -h production-server -U postgres -d mydb \
    --data-only --table=orders orders_recovered.dump
# OR: truncate orders first, then restore
```

Monitoring PITR Recovery Progress

```
-- Check recovery status
SELECT pg_is_in_recovery(),
       pg_last_wal_receive_lsn(),
       pg_last_wal_replay_lsn(),
       pg_last_xact_replay_timestamp();

-- How much WAL has been replayed
SELECT
    pg_last_xact_replay_timestamp() AS replayed_through,
    now() - pg_last_xact_replay_timestamp() AS lag_from_now;

-- Check which WAL files are being applied
-- (look in PostgreSQL log for "restored log file" messages)
sudo grep "restored log file" /var/log/postgresql/postgresql-16-main.log | tail -20
```

```
# Monitor archive WAL consumption during recovery
watch -n 2 'ls -lt /archive/ | head -20'
# Watch which files are being accessed
```

Partial Recovery Scenarios

Recover One Table

```
# 1. Recover full database on separate server to target time
# 2. Export only the affected table
pg_dump -h recovery-server -U postgres -d mydb -t orders -F c -f orders.dump

# 3. Restore to production
pg_restore -h production -U postgres -d mydb \
    --data-only --truncate --table=orders orders.dump
```

Recovery to Last Known Good Transaction

```
# Find the last good transaction from audit/app logs
# Example: last successful order was txid 5551234 at 14:30:55

# Set recovery target to that transaction
recovery_target_xid = '5551234'
recovery_target_inclusive = 'true'   # Include this transaction

# Or use named restore point (proactive PITR):
# Before risky operation:
SELECT pg_create_restore_point('before_migration_20240115');
# In recovery:
# recovery_target_name = 'before_migration_20240115'
```

Common Mistakes

1. **archive_command that always exits 0** — masking errors; archive seems OK but files aren't there
2. **Not testing PITR** — discovering it doesn't work during an actual incident
3. **archive_timeout too high** — RPO worse than expected (large data loss window)
4. **restore_command not matching archive_command** — wrong path, wrong compression
5. **Recovery target in wrong timezone** — off by hours
6. **Using standby.signal for pure PITR** — can cause incorrect behavior
7. **Not including inclusive vs exclusive** — off-by-one in recovery target
8. **Recovering in-place** — destroying production while recovering
9. **WAL archive not accessible from recovery server** — recovery fails
10. **Not verifying data before promoting** — promoting before confirming recovery worked

---

Best Practices

1. **Test PITR monthly** — recover to a test server and verify data
2. **Use `archive_timeout = 300`** for 5-minute RPO without needing manual WAL switches
3. **Archive to durable storage** (S3, GCS) — not just local disk
4. **Always recover on a separate server** — never overwrite production before verification
5. **Use `recovery_target_action = 'pause'`** to verify data before promoting
6. **Monitor `pg_stat_archiver`** continuously — alert on archive failures
7. **Set up both backup + WAL archiving** — pgBackRest handles both together
8. **Document recovery target syntax** — pre-write the recovery config template
9. **Keep the restore_command** script in version control
10. **Create named restore points** before risky operations (migrations, bulk updates)

Interview Questions

Q1: What is Point-in-Time Recovery and when do you use it?

A: PITR allows recovering a PostgreSQL database to any specific moment in time after the last base backup. It's used when a logical error occurs: accidental data deletion, incorrect bulk update, or application bug that corrupted data. Without PITR, you can only restore to the last backup time. With PITR + WAL archiving, you can recover to within minutes of the incident, minimizing data loss.

Q2: What are the two components required for PITR?

A: (1) A base backup (taken with pg_basebackup or pgBackRest) — provides the starting point for recovery. (2) Continuous WAL archive (`archive_command`) — provides the log of every database change from backup time to the recovery target. Together: restore base backup + replay archived WAL to target time = data as of that moment.

Q3: What is `archive_command` and what happens if it fails?

A: `archive_command` is a shell command PostgreSQL runs to copy each WAL segment to the archive. If it fails (non-zero exit code), PostgreSQL retries and does NOT overwrite or delete the WAL segment. It keeps retrying until success or a new segment is needed. If archiving falls significantly behind, WAL segments accumulate in `pg_wal/` . It's critical that archive_command failures are monitored via `pg_stat_archiver.failed_count` .

Q4: What is `recovery_target_time` and how do you use it?

A: `recovery_target_time` specifies the timestamp at which recovery should stop. During recovery, PostgreSQL applies WAL records one transaction at a time. When it encounters a transaction committed at or after `recovery_target_time` , it stops. This leaves the database in the state it was at that exact moment. Format: ISO 8601 with timezone: `'2024-01-15 14:31:00 UTC'` .

Q5: What is the difference between `recovery_target_action = pause` vs `promote` ?

A: With `pause` , PostgreSQL reaches the target time and stops applying WAL, but stays in recovery mode (read-only). You can connect and verify the data, then run `SELECT pg_wal_replay_resume()` to promote. With `promote` , it automatically becomes a primary as soon as it reaches the target. Use `pause` when you want to verify before committing to the recovery point; use `promote` for automated recovery where you trust the target time.

Q6: How does `archive_timeout` affect your RPO?

A: WAL segments are 16MB by default. A segment is only archived when it fills. For a low-activity database, segments might take hours to fill. `archive_timeout` forces a WAL switch every N seconds, ensuring segments are archived regularly. With `archive_timeout = 300` (5 minutes), the maximum gap between archived WAL segments is 5 minutes. This means your RPO is approximately 5 minutes, even during quiet periods.

Q7: What is a named restore point and how do you use it?

A: A named restore point is a user-defined label placed at a specific WAL position: `SELECT pg_create_restore_point('before_migration')` . This records the LSN. In recovery configuration: `recovery_target_name = 'before_migration'` . PostgreSQL replays WAL exactly to that named point. Useful before risky operations (migrations, bulk deletes) where you want a clean rollback point by name rather than timestamp.

Q8: How do you recover a single dropped table using PITR?

A: (1) Set up a recovery server. (2) Restore base backup from before the table drop. (3) Configure `recovery_target_time` to just before the DROP. (4) Start recovery — full database recovers to that time. (5) Use `recovery_target_action = 'pause'` to verify. (6) Export the specific table: `pg_dump -t dropped_table` . (7) Promote or stop recovery server. (8) Restore just that table to production.

Exercises and Solutions

Exercise 1: Set Up PITR from Scratch

```
#!/bin/bash
# setup_pitr.sh — Configure archiving and take first backup

# 1. Create archive directory
mkdir -p /var/pg_archive
chown postgres:postgres /var/pg_archive

# 2. Configure postgresql.conf
sudo -u postgres psql << 'EOF'
ALTER SYSTEM SET archive_mode = 'on';
ALTER SYSTEM SET archive_command = 'test ! -f /var/pg_archive/%f && cp %p
/var/pg_archive/%f';
ALTER SYSTEM SET archive_timeout = '300';
ALTER SYSTEM SET wal_level = 'replica';
EOF

# 3. Restart PostgreSQL (archive_mode requires restart)
sudo systemctl restart postgresql@16-main

# 4. Verify archiving
sudo -u postgres psql -c "SELECT * FROM pg_stat_archiver;"

# 5. Take first base backup
sudo -u postgres pg_basebackup \
    -D /var/backups/pg_pitr_base \
    --wal-method=stream \
    --checkpoint=fast \
    --progress \
    --verbose

echo "PITR is now configured!"
echo "Archive directory: /var/pg_archive"
echo "Base backup: /var/backups/pg_pitr_base"
```

Exercise 2: Practice PITR Recovery

```
# 1. Create test table and insert data
psql -U postgres -d testdb -c "
CREATE TABLE pitr_test (id SERIAL, data TEXT, ts TIMESTAMPTZ DEFAULT now());
INSERT INTO pitr_test (data) SELECT 'initial data ' || i FROM
generate_series(1,1000) i;
"

# 2. Record time and create restore point
psql -U postgres -d testdb -c "SELECT now() AS safe_time,
pg_create_restore_point('before_delete');"

# 3. "Accidentally" delete all rows
```

```
psql -U postgres -d testdb -c "DELETE FROM pitr_test;"
psql -U postgres -d testdb -c "SELECT COUNT(*) FROM pitr_test;"  # Shows 0

# 4. Force WAL archive
psql -U postgres -c "SELECT pg_switch_wal();"
sleep 2

# 5. Perform PITR to named restore point
# (Follow full PITR walkthrough from above using 'before_delete' as target)
```

Cross-References

- 05_wal_archiving.md — WAL archiving in depth
- 03_pg_basebackup.md — Base backup for PITR
- 06_pgbackrest.md — pgBackRest for automated PITR
- 07_disaster_recovery.md — DR scenarios using PITR
- 01_backup_strategies.md — PITR in backup strategy context